📘 JOBSHEET PRAKTIKUM

**Mata Kuliah: Pemrograman Framework**

**Pertemuan: Middleware & Route Protection**

**Topic: Proteksi Halaman dengan Middleware**

---

## 🎯 A. Capaian Pembelajaran (CPMK)

Mahasiswa mampu:

1. Menjelaskan konsep Middleware pada Next.js.
2. Membedakan redirect menggunakan useEffect dan Middleware.
3. Membuat file middleware.ts.
4. Mengatur proteksi route tertentu.
5. Mengimplementasikan sistem login sederhana menggunakan Middleware.

---

## 🧠 B. Dasar Teori

1⃣Apa itu Middleware?

Middleware adalah kode yang dijalankan sebelum request halaman diproses.

**Alur kerja:**

User akses halaman

↓

Middleware dijalankan

↓

Cek kondisi (login / role / dll)

↓

Lanjutkan request atau redirect

- Middleware berjalan di server/edge layer sebelum halaman dirender.

---

### 2⃣ Mengapa Tidak Menggunakan useEffect?

Redirect dengan useEffect:

```tsx
useEffect(() => {
  if (!isLogin) {
    router.push('/login')
  }
}, [])
```

Masalah:

- Halaman sempat terbuka dulu
- Terjadi "glitch"
- Kurang aman

Middleware:

- Redirect sebelum halaman dirender
- Tidak ada glitch
- Lebih secure

---

## 🛠️ C. Langkah Praktikum

---

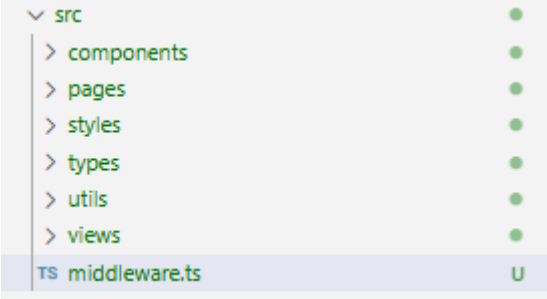
### 🔹 Bagian 1 – Membuat Middleware

- Modifikasi file index.tsx pada folder src/pages/produk

12.praktikum-Middleware > my-app > src > pages > produk > index.tsx > ...

```tsx
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "../../views/product";
import useSWR from "swr";
import fetcher from "../../utils/swr/fetcher";



// const fetcher = (url: string) => fetch(url).then((res) => res.json());
Windsurf: Refactor | Explain | ×
const kategori = () => {
  // const [isLogin, setIsLogin] = useState(true);
  const { push } = useRouter();
  const [products, setProducts] = useState([]);
  // console.log("products:", products);···

  const { data, error,isLoading } = useSWR("/api/produk", fetcher);
  //cek apakah data, error, dan isLoading sudah benar···


  return (
    <div>
      <TampilanProduk products={isLoading ? [] : data.data} />
    </div>
  );
};

export default kategori;

```

- Buat file: src/middleware.ts Sejajar dengan folder pages.




---

## Bagian 2 – Struktur Dasar Middleware

```
12.praktikum-Middleware > my-app > src > TS middleware.ts > middleware
1  import { NextResponse } from 'next/server'
2  import type { NextRequest } from 'next/server'
3
   Windsurf: Refactor | Explain | Generate JSDoc | X
4  export function middleware(request: NextRequest) {
5      return NextResponse.next();
6  }
```

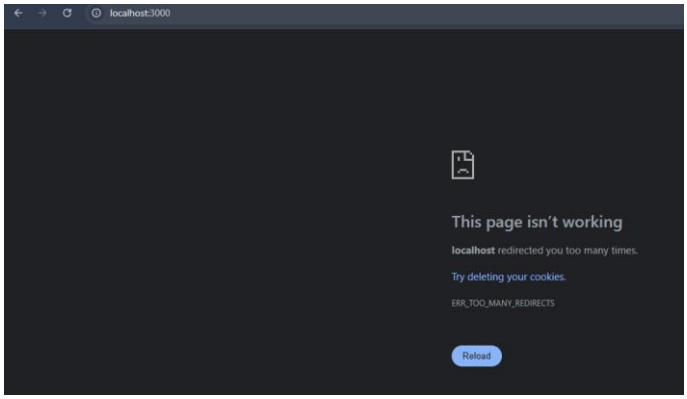
- Jika menggunakan NextResponse.next() → tidak ada redirect.
- Jadi masih bisa mengakses ke [http://localhost:3000/produk](http://localhost:3000/produk)

---

## Bagian 3 – Redirect Sederhana

```
12.praktikum-Middleware > my-app > src > TS middleware.ts > ...
1  import { NextResponse } from "next/server";
2  import type { NextRequest } from "next/server";
3
   Windsurf: Refactor | Explain | Generate JSDoc | X
4  export function middleware(request: NextRequest) {
5      return NextResponse.redirect(new URL("/", request.url));
6
7    //return NextResponse.next();
8  }
9
```

- Semua halaman akan redirect ke home dan error dikarenakan terus menerus loading



- **Bagian 4 – Batasi Route Tertentu**
  - Untuk mengatasi pada bagian 3 maka perlu pembatasan route

12.praktikum-Middleware > my-app > src > TS middleware.ts > ...

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

Windsurf: Refactor | Explain | Generate JSDoc | X
export function middleware(request: NextRequest) {
  return NextResponse.redirect(new URL("/", request.url));
  //return NextResponse.next();
}

export const config = {
  matcher: ["/produk", "/about"],
};

```

  - Artinya:
    - Middleware hanya berlaku untuk /products dan /about
    - Halaman lain tetap normal
    - Ketika user mengakses halaman produk dan about maka akan langsung redirect ke halaman home

- **Bagian 5 – Simulasi Sistem Login**
  - Modifikasi file middleware.ts

12.praktikum-Middleware > my-app > src > TS middleware.ts > ...

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

Windsurf: Refactor | Explain | Generate JSDoc | X
export function middleware(request: NextRequest) {
    const isLogin = false;
    if (isLogin) {
        return NextResponse.next();
    } else {
        return NextResponse.redirect(new URL("/auth/login", request.url));
    }

    //return NextResponse.redirect(new URL("/", request.url));
  //return NextResponse.next();
}

export const config = {
  matcher: ["/produk", "/about"],
};

```

  - Jika user langsung mengakses ke alamat http://localhost:3000/produk tidak akan bisa user akan diarahkan ke halaman login

---

## 🔬 D. Pengujian

---

### 🧪 Uji 1 – isLogin = false

Akses:

/products

Hasil:
➡️ Redirect ke /login

---

### 🧪 Uji 2 – isLogin = true

Ubah:

```
const isLogin = true
```

Hasil:
➡️ Bisa mengakses /products

---

### 🧪 Uji 3 – Tambahkan Multiple Route

```
export const config = {
  matcher: ['/products', '/about']
}
```

Sekarang:

- /products dan /about butuh login
- Halaman lain bebas

---

## 📊 E. Perbandingan Middleware vs useEffect

| Aspek | useEffect | Middleware |
|---|---|---|
| Redirect timing | Setelah render | Sebelum render |
| Glitch | Ada | Tidak |
| Security | Lemah | Lebih aman |
| Skalabilitas | Harus tiap halaman | Sekali di middleware |

---

## 🧪 F. Tugas Praktikum

🎯 Tugas Individu

1. Buat halaman:
    - /products
    - /about
    - /login
2. Implementasikan Middleware:
    - Redirect ke /login jika belum login.
    - Izinkan akses jika login true.
3. Tambahkan proteksi hanya untuk route tertentu.
4. Dokumentasikan:
    - Screenshot sebelum dan sesudah redirect.
    - Perbandingan dengan useEffect.

---

## 📝 G. Pertanyaan Analisis

1. Mengapa middleware lebih aman dibanding useEffect?
2. Mengapa middleware tidak menimbulkan glitch?
3. Apa risiko jika semua halaman diproteksi tanpa pengecualian?
4. Kapan middleware tidak diperlukan?
5. Apa perbedaan middleware dan API route?